

14-mavzu. Texnik tizimlardagi hisoblash masalalari. Chiziqli dasturlash masalalari. Jarayonlarning chiziqli modellari tuzish

Reja

1. Ma'lumotlarning faylli toifasi, ularning turlari, ularga murojaat qilish.
2. Matnli fayllar bilan ishlash.
3. Fayllar ustida turli amallar.
4. Ularni texnik yo'nalishdagi masalalarda ishlatilishi.
5. Fayllarni dasturlarda qo'llash.

Python texnik yo'nalishlarda (elektronika, mexanika, elektrotexnika, avtomatika, ishlab chiqarish) hisoblash va modellashtirish uchun eng qulay tillardan biri hisoblanadi. Chunki Python sodda sintaksisga ega, kuchli kutubxonalari (masalan, numpy, pandas, scipy) mavjud va amaliy masalalarda tez natija beradi. Texnik tizimlarda esa doimiy ravishda ma'lumot yig'iladi: sensor ko'rsatkichlari, tajriba natijalari, o'lchov jadvallari, qurilma sozlamalari va boshqalar. Shu ma'lumotlar ko'pincha **fayllar** ko'rinishida saqlanadi va dasturlar orqali qayta ishlanadi.

Python'da fayllar bilan ishlashni bilish — texnik masalalarni avtomatlashtirishning "kaliti". Masalan, chiziqli model (jarayonni taxminan to'g'ri chiziqli bilan ifodalash) tuzish uchun ko'plab (x, y) tajriba nuqtalarini fayldan o'qib olamiz, model parametrlari (a va b) ni hisoblaymiz va natijani faylga yozamiz. Chiziqli dasturlash (Linear Programming) masalalarida ham koeffitsiyentlar, cheklovlar va natijalar faylda saqlanib, dasturga yuklanadi. Natijada hisoblash jarayoni tezlashadi, xatolik kamayadi va tizimni tahlil qilish osonlashadi.

1. Ma'lumotlarning faylli toifasi, turlari, ularga murojaat qilish (Python'da)

Python'da fayl — bu diskda saqlanadigan ma'lumot manbai bo'lib, dastur uni o'qishi (read) yoki yozishi (write) mumkin. Texnik hisoblashlarda "faylli ma'lumotlar" deganda asosan tajriba natijalari, sensor loglari, hisoblash natijalari va konfiguratsiya parametrlarini tushunamiz. Python'da fayllar bilan ishlashning eng asosiy mexanizmi open() funksiyasi orqali amalga oshiriladi. Fayl ochilganda rejim tanlanadi: o'qish ("r"), yozish ("w"), qo'shib yozish ("a"), ba'zan ikkilik rejim ("rb", "wb").

Fayl turlari texnik masalalarda turlicha vazifa bajaradi. Matnli fayl (.txt, .csv) — koeffitsiyentlar va jadval ko'rinishidagi ma'lumotlar uchun qulay. CSV ayniqsa muhim: u Excel va ko'plab dasturlar bilan mos keladi. Ikkilik fayl (.bin, .dat) esa katta ma'lumotni tez saqlash uchun ishlatiladi (masalan, sensor oqimlari yoki qurilma loglari). Bundan tashqari, JSON yoki YAML kabi formatlar qurilma sozlamalarini saqlashda keng tarqalgan, chunki tuzilmalangan ma'lumot beradi.

Python'da faylga murojaat qilishda eng to'g'ri usul — with open(...) as f: konstruktsiyasidan foydalanish. Bu usul faylni avtomatik yopadi va texnik tizimlarda juda muhim bo'lgan "resursni unutib qo'yish" muammosini oldini oladi. Umuman, texnik hisoblashlarda faylga murojaat qilish — bu ma'lumotlarni boshqarish, tahlil qilish va keyingi modellashtirishga tayyorlashning fundamental bosqichidir.

2. Matnli fayllar bilan ishlash (Python'da TXT va CSV)

Matnli fayllar Python'da eng ko'p qo'llaniladigan formatdir. Texnik yo'nalishlarda matnli fayl odatda ikki xil ko'rinishda uchraydi: oddiy .txt va jadval ko'rinishidagi .csv. .txt fayl ko'pincha oddiy ro'yxatlar, parametrlar (masalan, "R=10, U=5") yoki log yozuvlari uchun ishlatiladi. .csv esa tajriba natijalari jadvali (masalan, vaqt–kuchlanish, tezlik–vaqt) uchun juda qulay, chunki ustun/qatordan iborat tuzilmani saqlaydi.

Python'da .txt faylni o'qishda eng ko'p ishlatiladigan yo'l — read() (hammasini o'qish), readline() (bitta qator), readlines() (qatorlar ro'yxati) yoki fayl bo'ylab for line in f: qilib yurishdir. Texnik masalalarda aynan qatorlab o'qish foydali: masalan, har bir qator — bitta o'lchov natijasi bo'lishi mumkin. .csv bilan ishlashda esa csv moduli yoki amaliyotda ko'proq pandas ishlatiladi. pandas.read_csv() birgina funksiya bilan katta jadvalni o'qib, darhol tahlil qilish imkonini beradi.

Matnli fayllarning eng katta afzalligi — tekshirish va tahrirlash osonligi. Masalan, laboratoriya ishlari natijasini .csv da saqlab, keyin Python'da grafik chizish mumkin. Yoki chiziqli model tuzishda tajriba nuqtalarini .txt ga yozib, dasturga yuklab, $y = ax + b$ parametrlarini hisoblab natijani faylga chiqarish mumkin.

Kamchiligi ham bor: juda katta ma'lumotlarda matnli format sekinroq va ko'proq joy egallaydi. Lekin o'quv va ko'p amaliy masalalarda .txt/.csv eng qulay format bo'lib qoladi.

3. Fayllar ustida turli amallar (Python'da amaliy yondashuv)

Python'da fayllar bilan ishlash faqat o'qish/yozish emas, balki faylni yaratish, nomini o'zgartirish, mavjudligini tekshirish, o'chirish, papkalar bilan ishlash kabi amallarni ham o'z ichiga oladi. Texnik tizimlarda bu amallar juda amaliy: masalan, har bir tajriba kuni uchun alohida log fayl yaratish, eskilarini arxivlash, natijalarni alohida papkaga ajratish.

Faylni yaratishning oddiy usuli — "w" rejimida ochish. Agar fayl yo'q bo'lsa, Python uni yaratadi. "a" rejimi esa mavjud fayl oxiriga ma'lumot qo'shadi, bu ayniqsa sensor loglari uchun muhim: tizim ishlayveradi, faylga yangi satrlar qo'shib boradi. "r" rejimida o'qish esa ma'lumot tahliliga kirish bosqichidir.

Fayl ustida “xavfsiz ishlash” texnik muhandislikda muhim. Masalan, noto‘g‘ri fayl nomiga yozib yuborish natijada muhim ma’lumotni yo‘qotishi mumkin. Shu sabab, Python’da ko‘pincha fayl mavjudligini tekshirish (`os.path.exists`) va ehtiyot nusxa yaratish amaliyoti qo‘llanadi. Faylni o‘chirish (`os.remove`), nomini o‘zgartirish (`os.rename`) yoki papka yaratish (`os.makedirs`) kabi amallar tajriba natijalarini tartibli saqlashga yordam beradi.

Yana bir muhim jihat — kodda xatolik bo‘lsa ham fayl yopilib ketishi. `with open(...)` as `f`: konstruktsiyasi aynan shuni kafolatlaydi. Texnik hisoblashlarda resurslar (fayl, port, qurilma) doim boshqaruv ostida bo‘lishi kerak. Fayl bilan ishlash madaniyati to‘g‘ri bo‘lsa, keyingi bosqich — modellash va optimallashtirish (chiziqli dasturlash) ancha ishonchli ishlaydi.

4. Fayllarni texnik yo‘nalishdagi masalalarda ishlatilishi (Python misolida)

Texnik yo‘nalishlarda fayllar amalda “real jarayon” va “hisoblash dasturi” orasidagi ko‘prik vazifasini bajaradi. Masalan, elektronika laboratoriyasida o‘lchangan tok va kuchlanish qiymatlari Excel/CSV ko‘rinishida saqlanadi. Python dasturi bu faylni o‘qib, filtrlaydi, o‘rtacha qiymat oladi, grafik chizadi yoki model moslashtiradi. Mexanikada ham shunday: tezlik, kuch, tebranish amplitudasi kabi ma’lumotlar fayllarda yig‘iladi.

Jarayonlarning chiziqli modellari ham ko‘pincha fayldagi tajriba nuqtalaridan olinadi. Masalan, (x, y) nuqtalar berilgan bo‘lsa, Python’da regressiya usuli orqali $y = ax + b$ ko‘rinishidagi chiziqli model topiladi. Bu yerda fayl — ma’lumot manbai, Python esa hisoblovchi va tahlil qiluvchi vosita.

Chiziqli dasturlash masalalarida faylning roli yanada aniq: koeffitsiyentlar, cheklovlar va resurslar odatda jadval ko‘rinishida bo‘ladi. Misol: ishlab chiqarishda xomashyo cheklovi, vaqt cheklovi, talab cheklovi — ularni CSV faylga yozib qo‘yish mumkin. Python dasturi fayldan o‘qib, optimallashtirishni bajaradi va maksimal foyda yoki minimal xarajatga erishish bo‘yicha yechimni chiqaradi.

Avtomatik boshqaruvda esa log fayllar juda muhim: tizimdan kelayotgan signallar, xatolik kodlari, sensor qiymatlari faylga yozilib boradi. Python bu loglarni tahlil qilib, nosozlik sababini topishda yordam beradi. Shunday qilib, fayllar texnik masalalarda saqlash emas, balki tahlil va optimallashtirishning ham asosiy komponentidir.

5. Fayllarni dasturlarda qo‘llash (Python’da chiziqli model va LP bilan bog‘lash)

Python’da fayllardan foydalanish texnik hisoblashlarni avtomatlashtirishning eng kuchli usullaridan biridir. Amaliyotda jarayon quyidagicha bo‘ladi: avval ma’lumotlar (tajriba natijalari, sensor oqimi yoki koeffitsiyentlar) faylda saqlanadi,

so'ng Python dasturi ularni o'qib, kerakli hisob-kitobni bajaradi va natijani qaytadan faylga chiqaradi. Bu yondashuv "qo'lda kiritish" muammosini bartaraf etadi, demak xatolik kamayadi.

Chiziqli model tuzishda fayl orqali ishlash ayniqsa qulay. Masalan, ustunlarda x va y bo'lgan CSV fayl bo'lsin. Python uni o'qib, a va b ni topadi (eng sodda holatda eng kichik kvadratlar g'oyasi bilan). Natijada siz jarayonning taxminiy tenglamasini olasiz: $y = ax + b$. Bu tenglama texnik tizimni tushuntirish, bashorat qilish va boshqaruv algoritmini sozlashda ishlatiladi.

Chiziqli dasturlash (LP) masalalariga kelganda, fayl juda amaliy: masalan, maqsad funksiyasi koeffitsiyentlari alohida qatorda, cheklovlar esa jadval ko'rinishida bo'ladi. Python bu fayldan o'qib, masalani matematik ko'rinishga keltiradi va yechadi (amaliyotda `scipy.optimize` yoki boshqa paketlar ishlatiladi). Natijalar — optimal qarorlar (qaysi resursdan qancha ishlatish, qaysi mahsulotdan qancha ishlab chiqarish) — yana faylga yoziladi. Bu natijani keyin Excel'da ko'rish yoki hisobotga qo'shish mumkin.

Xulosa qilib aytganda, Python'da fayllar bilan ishlash — modellashtirish, optimallashtirish va texnik tahlilning ajralmas qismidir.

1-masala: Fayl turlari va ularga murojaat qilish (open, rejimlar)

Kichik izoh:

Python'da fayl bilan ishlash `open()` orqali boshlanadi. Rejimlar:

- "r" — o'qish,
 - "w" — yangidan yozish (eski faylni tozalab yuboradi),
 - "a" — oxiriga qo'shib yozish.
- Eng to'g'ri usul — with `open(...)` as `f`: (fayl avtomatik yopiladi).

1) Faylga yozish (w) va o'qish (r)

```
filename = "params.txt"
```

Texnik parametrlar (masalan: R - qarshilik, U - kuchlanish)

with `open(filename, "w", encoding="utf-8")` as `f`:

```
f.write("R=10\n")
```

```
f.write("U=5\n")
```

```

f.write("I=0.5\n")

# Fayldan o'qish
params = {}

with open(filename, "r", encoding="utf-8") as f:
    for line in f:
        key, val = line.strip().split("=")
        params[key] = float(val)

print("O'qilgan parametrlar:", params)
print("Tekshiruv: U/R =", params["U"] / params["R"])

```

2-masala: Matnli fayllar bilan ishlash (TXT va CSV)

Kichik izoh:

TXT — oddiy satrlar; CSV — jadval (ustun-qator). Texnik tajribada ko‘pincha vaqt–kuchlanish kabi jadval bo‘ladi. CSV’ni **csv** moduli bilan oson boshqaramiz.

2) CSV yaratish va o‘qish: (t, U) jadvali
import csv

```
csv_file = "measurements.csv"
```

Tajriba: vaqt (s) va kuchlanish (V)

```

data = [
    (0.0, 0.10),
    (0.5, 0.45),
    (1.0, 0.95),
    (1.5, 1.40),
    (2.0, 2.05),
]

```

CSV ga yozish

```

with open(csv_file, "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["t_sec", "U_volt"])
    writer.writerows(data)

```

```

# CSV dan o'qish
t_values, u_values = [], []
with open(csv_file, "r", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        t_values.append(float(row["t_sec"]))
        u_values.append(float(row["U_volt"]))

print("t:", t_values)
print("U:", u_values)
print("O'rtacha U:", sum(u_values) / len(u_values))

```

3-masala: Fayllar ustida turli amallar (mavjudligini tekshirish, rename, delete, papka)

Kichik izoh:

Texnik loyihada natijalarni tartibli saqlash uchun papkalar, nomlash, arxivlash muhim. pathlib (tavsiya) fayl/papka amallarini qulay qiladi.

3) Fayl/papka amallari: exists, mkdir, rename, delete

```

from pathlib import Path
import shutil

```

```

base_dir = Path("results")
base_dir.mkdir(exist_ok=True) # papka yaratish

```

```

src = Path("measurements.csv")
dst = base_dir / "measurements_backup.csv"

```

Fayl mavjudligini tekshirish va nusxa ko'chirish

```

if src.exists():
    shutil.copy(src, dst)
    print("Nusxa olindi:", dst)
else:
    print("Topilmadi:", src)

```

Fayl nomini o'zgartirish (rename)

```

renamed = base_dir / "meas_01.csv"
if dst.exists():
    dst.rename(renamed)
    print("Nom o'zgardi:", renamed)

```

Faylni o'chirish (delete) - ehtiyot bo'ling!

```

# if renamed.exists():

```

```
# renamed.unlink()
# print("O'chirildi:", renamed)
```

4-masala: Texnik masalalarda fayllardan foydalanish (log yozish, sensor oqimi)

Kichik izoh:

Avtomatika/elektronikada sensor qiymatlari vaqt bo'yicha keladi. "a" rejimi bilan log faylga qo'shib boriladi. Keyin log o'qilib, masalan, limitdan oshgan holatlar topiladi.

4) Sensor log yozish va tahlil qilish

```
import random
from datetime import datetime
```

```
log_file = "sensor_log.txt"
```

1) Logga qo'shib yozish (append)

```
with open(log_file, "a", encoding="utf-8") as f:
```

```
    for _ in range(10):
```

```
        ts = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```
        temp = round(random.uniform(20.0, 80.0), 2) # masalan, harorat
```

```
        f.write(f"{ts};TEMP={temp}\n")
```

```
print("Log yozildi:", log_file)
```

2) Logdan o'qib, kritik holatlarni topish (masalan TEMP>60)

```
critical = []
```

```
with open(log_file, "r", encoding="utf-8") as f:
```

```
    for line in f:
```

```
        line = line.strip()
```

```
        if not line:
```

```
            continue
```

```
        ts_part, temp_part = line.split(";")
```

```
        temp_val = float(temp_part.split("=")[1])
```

```
        if temp_val > 60:
```

```
            critical.append((ts_part, temp_val))
```

```
print("Kritik holatlar (TEMP>60):")
```

```
for ts, val in critical:
```

```
    print(" -", ts, val)
```

5-masala: Fayllarni dasturlarda qo'llash (chiziqli model: $y = ax + b$)

Kichik izoh:

Jarayonni chiziqli model bilan yaqinlashtirish texnikada ko'p: masalan,

“kuchlanish–vaqt”, “kuch–deformatsiya”, “tok–kuchlanish”. Fayldan (x, y) nuqtalarni o‘qib, a va b ni hisoblaymiz va natijani faylga yozamiz. Quyida oddiy **eng kichik kvadratlar** formulasi bilan.

5) CSV dan (x,y) ni o‘qib, $y = a*x + b$ ni topish va natijani yozish
import csv

input_csv = "xy_data.csv"
output_txt = "linear_model_result.txt"

(x, y) ma’lumotlar yaratib qo‘yamiz (tajriba nuqtalari)

```
points = [  
    (0.0, 1.0),  
    (1.0, 2.1),  
    (2.0, 2.9),  
    (3.0, 4.2),  
    (4.0, 5.1),  
]
```

```
with open(input_csv, "w", newline="", encoding="utf-8") as f:  
    w = csv.writer(f)  
    w.writerow(["x", "y"])  
    w.writerows(points)
```

CSV dan o‘qish

```
xs, ys = [], []  
with open(input_csv, "r", newline="", encoding="utf-8") as f:  
    r = csv.DictReader(f)  
    for row in r:  
        xs.append(float(row["x"]))  
        ys.append(float(row["y"]))
```

Eng kichik kvadratlar (a, b) hisoblash

```
n = len(xs)  
x_mean = sum(xs) / n  
y_mean = sum(ys) / n
```

```
num = sum((xs[i] - x_mean) * (ys[i] - y_mean) for i in range(n))  
den = sum((xs[i] - x_mean) ** 2 for i in range(n))
```

```
a = num / den  
b = y_mean - a * x_mean
```

Natijani faylga yozish

```
with open(output_txt, "w", encoding="utf-8") as f:
```



```

f.write(f"Linear model:  $y = a \cdot x + b$ \n")
f.write(f"a = {a:.6f}\n")
f.write(f"b = {b:.6f}\n")

print("Model topildi: y =", round(a, 4), "* x +", round(b, 4))
print("Natija yozildi:", output_txt)

# Tekshiruv: biror x uchun bashorat
x_test = 2.5
y_pred = a * x_test + b
print(f"x={x_test} uchun bashorat y={y_pred:.3f}")

```

Nazorat savollari

1. Python'da fayl ochish uchun qaysi funksiya ishlatiladi va nima uchun with ... as ... tavsia qilinadi?
2. "r", "w", "a" rejimlari nimani anglatadi?
3. Matnli fayl va ikkilik faylning farqi nimada, qaysi holatda qaysi biri qulay?
4. .csv fayl texnik hisoblashlarda nima uchun ko'p ishlatiladi?
5. Faylni yopmaslik qanday muammolarga olib kelishi mumkin?
6. Fayl mavjudligini tekshirish nimaga kerak (texnik misol bilan)?
7. Sensor loglarini Python'da saqlash uchun qaysi fayl rejimi ko'proq mos?
8. Jarayonning chiziqli modeli $y = ax + b$ ni tuzishda fayllar qanday rol o'ynaydi?
9. Chiziqli dasturlash masalasi koeffitsiyentlarini faylda saqlashning afzalliklari nimalar?
10. Faylga yozilgan natijalar keyingi tahlil va hisobotda qanday foyda beradi?